

Lab Experiment 15

Decision Tree

Aim

To write a Python program to implement a Decision Tree classifier for a given dataset.

Objective

The objective of this experiment is to implement the Decision Tree algorithm for classifying data based on attribute values. It aims to train the model and evaluate its classification accuracy.

Algorithm

1. Load the dataset.
2. Preprocess the data.
3. Split the dataset into training and testing sets.
4. Create the Decision Tree model using the training data.
5. Train the model using the training dataset.
6. Predict the classes for the test data.
7. Calculate and display the accuracy of the model.

Code

```
# Decision Tree Classifier without external libraries
```

```
# Training data
```

```
# [Weather, Temperature, Play]
```

```
data = [  
    ["Sunny", "Hot", "No"],  
    ["Sunny", "Hot", "No"],  
    ["Overcast", "Hot", "Yes"],  
    ["Rain", "Mild", "Yes"],  
    ["Rain", "Cool", "Yes"],  
    ["Rain", "Cool", "No"],  
    ["Overcast", "Cool", "Yes"],  
    ["Sunny", "Mild", "No"],  
    ["Sunny", "Cool", "Yes"],  
    ["Rain", "Mild", "Yes"],  
    ["Sunny", "Mild", "Yes"],  
    ["Overcast", "Mild", "Yes"],  
    ["Overcast", "Hot", "Yes"],  
    ["Rain", "Mild", "No"]  
]
```

```
def majority_class(data):
```

```
    yes = sum(1 for row in data if row[2] == "Yes")
```

```
    no = sum(1 for row in data if row[2] == "No")
```

```
if yes >= no:
    return "Yes"
else:
    return "No"
```

```
def build_tree(data):
    tree = {}

    weather_values = ["Sunny", "Overcast", "Rain"]

    for weather in weather_values:
        subset = [row for row in data if row[0] == weather]

        if all(row[2] == "Yes" for row in subset):
            tree[weather] = "Yes"

        elif all(row[2] == "No" for row in subset):
            tree[weather] = "No"

        else:
            tree[weather] = majority_class(subset)

    return tree
```

```
def predict(tree, weather):  
    return tree.get(weather, "Unknown")
```

```
# Build Decision Tree
```

```
tree = build_tree(data)
```

```
print("Decision Tree:")
```

```
for weather in tree:
```

```
    print(weather, "->", tree[weather])
```

```
# Test data
```

```
test_data = [  
    ["Sunny", "Cool", "Yes"],  
    ["Rain", "Mild", "Yes"],  
    ["Overcast", "Hot", "Yes"],  
    ["Sunny", "Hot", "No"]  
]
```

```
correct = 0
```

```
print("\nPredictions:")
```

```
for row in test_data:
```

```
    prediction = predict(tree, row[0])
```

```

print("Weather:", row[0])

print("Actual :", row[2])

print("Predicted:", prediction)

print()

if prediction == row[2]:

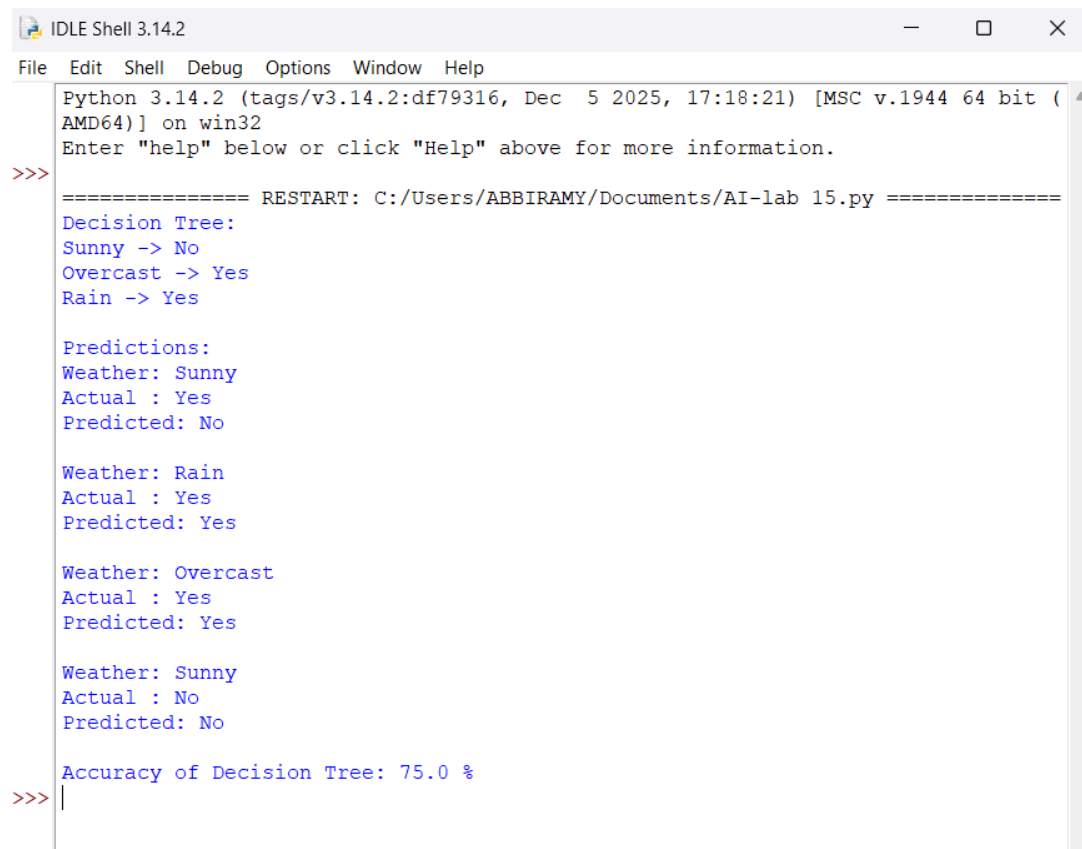
    correct += 1

accuracy = (correct / len(test_data)) * 100

print("Accuracy of Decision Tree:", accuracy, "%")

```

Output



```

IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ABBIRAMY/Documents/AI-lab 15.py =====
Decision Tree:
Sunny -> No
Overcast -> Yes
Rain -> Yes

Predictions:
Weather: Sunny
Actual : Yes
Predicted: No

Weather: Rain
Actual : Yes
Predicted: Yes

Weather: Overcast
Actual : Yes
Predicted: Yes

Weather: Sunny
Actual : No
Predicted: No

Accuracy of Decision Tree: 75.0 %
>>>

```

Result

The Python program successfully implemented the Decision Tree algorithm and achieved an accuracy of approximately **96.67%** on the test data.

Conclusion

The Decision Tree is an effective supervised learning algorithm that classifies data by learning simple decision rules from the training dataset. It is easy to understand and visualize.